FC with RG – Novel HPC Gaudi 3

Jonathan Gan, Sunny Lin, Joseph Miao, Jeremy Wang, Keegan Williams

Background and Project Goals

Intel Gaudi 3: One of the newest AI accelerators that is more cost efficient than the NVIDIA H100s.

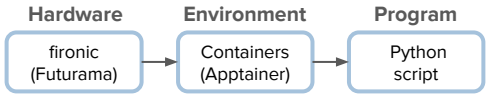
Original goal is to stress test and establish performance baselines using different AI models with up to 8 cards

Environment and Workflow

Onboarded new members with the example.py script and taught members how to allocate resources on fironic, run multi-card configuration through Deepspeed

Transitioned from venv to apptainers to ensure environment portability and define custom, shared command-line arguments, automating setup for new users.

Created standardized tutorials on how to make apptainer sandboxes



Methodology and Results

Optimum-habana Runscript

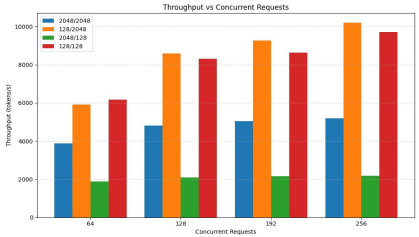
- Optimum-Habana's gaudi_spawn.py (with DeepSpeed) to launch run_generation.py across 1-8 HPUs using the Llama-2-70B-chat model.
- Key settings that control precision and performance: bf16, KV cache, HPU graphs, and workload shape (batch_size, max_input_tokens, max_new_tokens) for throughput benchmarking.

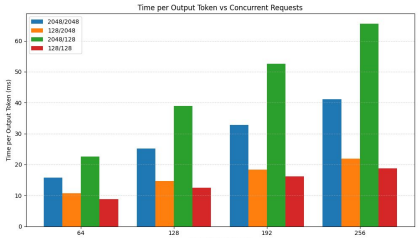
Intel's benchmark container

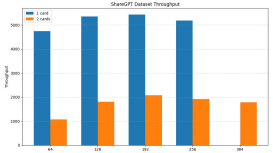
- Optimum Habana, transformers, and tokenizers version mismatch so not able to output results
- Intel's container might be outdated even though they changed the workflow recently

vLLM

- Llama 3.1 8B Instruct, 1 card
- Stress-testing compute
- Varying max concurrent requests (64, 128, 192, 256)
- Theoretical max req: 196.68
- Varying input/output token lengths (128 and 2048)
- Multi-card scaling: need to use other models







Discussion

A single Gaudi 3 chip running a small model like Llama 3.1 8B scales well with throughput as concurrent requests increase, with compute saturating around **5200 tok/s** for 2048/2048

As one might expect, latency steadily rises as concurrent requests and throughput increase

Variable-length inputs and outputs on ShareGPT dataset see a peak **5430 tok/s**

vLLM doesn't directly support data parallelism

Lessons Learned and Next Steps

Challenges and Lessons Learned

- Overflow error: resolved by updating drivers to v1.22.1
- Model missing tokenizers: resolved by redownloading in new directory

Next Semester Goals

- Official Gaudi docs can be outdated: Make sure to keep our team's internal docs updated
- vLLM - use more standardized configurations, run bigger models (Llama 3.1 405B), try FP8
- Reassess scalability analysis with new understanding

FC with RG – Novel HPC Power Analysis

Ibhan Aggarwal, Vishnu Kunnummal, Bryson Bennett, Lucas Goldfarb

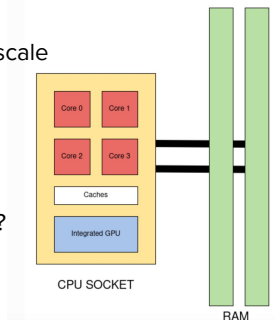
Background

CPU based inference offers a more **energy conscious** and **cost effective** alternative to GPU deployment for LLM workloads.

Looking at more **realistic use cases** for LLMs:

- How does inference scale with *longer* prompts?

- What is the effect of *frequent* request rates?

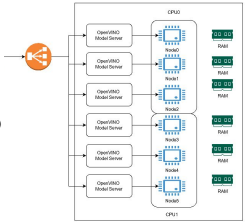


Goals/Planning/Milestones

- Evaluate the **power/energy consumption** of Intel Xeon Max Granite Rapids CPUs across inference params (input length, request rate)
- Transition to OpenVINO to take advantage of **optimizations** on Intel CPUs
- Compare findings to existing results and analyze trends

Environment and Setup

- Start up 6 OpenVino servers mounted on relevant models
- NGINX **load balancer** exposed in front of servers
- vLLM benchmarking setup to simulate realistic loads
- Used PyJoules wrapper to measure energy consumption



Methodology and Results

- PyJoules accesses RAPL to sample energy counters of **CPU sockets and DRAM** every second to poll the change in energy consumed
- Idle power draw measured for first 10 seconds
- Varied **input tokens (8 to 2048)** and **prompt request rate (8 to 256)** while measuring token throughput and energy consumption per token

- Throughput peaked at around **4750 tok/s** with input length of **2048** tokens, while energy consumption dropped to **180 J/tok**
- Both Mistral and Llama models with **FP16 quantizations** acted similarly across the board

Discussion

Input Length:
- Throughput increases with input length but tapers off
- Energy efficiency improves dramatically at longer inputs

Request Rate:
- Throughput and energy efficiency remains relatively stable across request rates

Node Comparison:
- MR-DIMM (phantasmic 1) and DDR5+CXL (phantasmic 2) perform nearly identically, minimal CXL overhead
- DDR5 (fironic) shows ~15-20% lower throughput and higher energy consumption across all conditions

Conclusion and Next Steps

- Developed a benchmarking workflow in order to get power statistics from LLM inference
- Analyzed trends in CPU only inference as they relate to power consumption

- Compare results to GPU-only setups
- Explore inference on **heterogeneous architectures** to achieve a better balance between power consumption and performance
- Test other workflows (e.g., image generation, reinforcement learning)

FC with RG – LLM Inference Optimization

Miles Kent, Jackson Belanger

Background and Project Goals

Implement symmetric quantization upon eviction of cold KV Cache from VRAM under vLLM inference engine, using LMCache as a KV Cache broker

Environment and Workflow

Cluster Hardware:

- NVIDIA H100 nodes on Futurama cluster

Inference Stack:

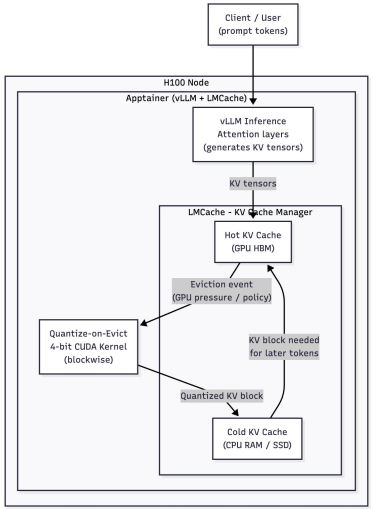
- vLLM as inference engine
- LMCache as broker for KV cache storage, access, and eviction
- CUDA for kernel dev

Dev tools:

- Apptainer for reproducible setup for inference across machines (contains full environment / dependencies)

Methodology and Results

- Evaluated various options for inference engines including implement a toy inference engine, llama.cpp, TensorRT, and vLLM
- Ran Llama 3.1 Instruct (8B parameters) using llama.cpp, and found it does not cleanly expose KV block logic in a way that we can effectively implement quantize-on-evict
- Evaluated inference stacks and selected vLLM + LMCache for having a clean, modular layer for KV cache logic
- Deployed vLLM + LMCache on H100 node and confirmed working model loading and inference (using Llama 3.1 8B)
- Built an Apptainer image containing vLLM, LMCache, CUDA toolkits / versions, and other dependencies, for quick reproducible runs



Discussion

- KV cache movement is the main bottleneck in long-context inference. We are targeting the right area
- Framework choice matters a ton. Limitations like llama.cpp not exposing KV block logic cleanly makes our intended optimizations unfeasible
- LMCache's modular design makes our cache-level policy feasible to implement

Conclusion and Next Steps

- Laid foundation for our new subteam by researching the problem, evaluating different frameworks / inference stacks, and implementing a reproducible image to run our stack

Next semester:

- Study LMCache's framework for handling KV blocks
- Implement + integrate symmetric quantization into LMCache's eviction path
- Benchmark baseline vs quantize-on-evict policy on long-context inference